

Challenge Write-Up Template

Challenge Information

Student Name	Pascua, Diana KC S.
Section / Class	BSIT-3G
Challenge Name	Undo
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/14/26
Flag Format Given	picoCTF{....}

1. Challenge Description

This challenge, Instead of a file to decode on my own, this one gave me an interactive server to connect to. Once connected, it walked me through five separate steps, one at a time. At each step it showed me the flag in its current (scrambled) form, told me in plain English what transformation had just been done to it, and asked me to type in the Linux command that would undo that exact transformation. If I got the command right, it moved on to the next step; the goal was to get through all five steps and land back on the original, readable flag.

2. Initial Analysis / Reconnaissance

The first thing I did was connect to the server with `nc foggy-cliff.picoctf.net 49918` to see what it actually wanted from me. As soon as I connected, it introduced itself as the "Text Transformations Challenge" and explained the rules: at every step it would show a transformed flag plus a hint describing what was done to it, and I had to answer with the Linux command that reverses that specific transformation. Because the challenge was explicitly telling me what each transformation was, this wasn't really a guessing game — it was a test of knowing the right Linux/shell command for each named operation (Base64, reversing text, character substitution, ROT13), so I filed this under general command-line / text-processing skills rather than anything cryptographic in a deeper sense.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
nc foggy-cliff.picoctf.net 49918	Connected to the challenges server to start the interactive session	Server greeted me with the challenge intro and immediately showed step1 with a transformed flag and a hint
Base64 -d	Reversed step 1, since the server told me the string had been base64 encoded	Server accepted it and moved on to step 2
rev	Reversed step 2, since the server told me the text had simply been reversed	Server accepted it and moved on to step 3
tr'-' " _ "	Reversed step 3, since the server told me underscores had been swapped for dashes	Server accepted it and moved on to step 4
tr'()' '{' '}'	Reversed step 4, since the server told me curly braces had been swapped for parenthesis	Server accepted it and moved on to step 5
tr 'A-Za-z' 'N-ZA-Mn-za-m'	Reversed step 5, since the server told me ROT13 had been applied to the letters	Server confirmed this was correct and printed the fully recovered flag

4. Solution Walkthrough

Step 1

Undo the Base64 encoding

The server showed a Base64-looking string and told me flat out: "Base64 encoded the string." So the fix was easy — the command that decodes Base64 in Linux is `base64 -d`. I entered it and got "Correct!", moving on to Step 2.

```
=====
PascuaDiana-academy@webshell:~$ nc foggy-cliff.picoctf.net 49918
===Welcome to the Text Transformations Challenge!===

Your goal: step by step, recover the original flag.
At each step, you'll see the transformed flag and a hint.
Enter the correct Linux command to reverse the last transformation.

--- Step 1 ---
Current flag: KTBxcDI0bnIwLWZhMDFnQHplMHnMYRlRy1nazNnLXRhMWZlcmlyRShTR1BicHZj
Hint: Base64 encoded the string.
Enter the Linux command to reverse it: base64 -d
Correct!
```

Step 2

Undo the text reversal

This time the hint said "Reversed the text." Reversing already-reversed text just un-reverses it, so I used `rev`, which flips a line of text back to front. That was accepted too.

```
--- Step 2 ---
Current flag: )0qp24nr0-fa01g@ze0sfa4eG-gk3g-ta1ferirE(SGPbpvc
Hint: Reversed the text.
Enter the Linux command to reverse it: rev
Correct!
```

Step 3

Undo the underscore/dash swap

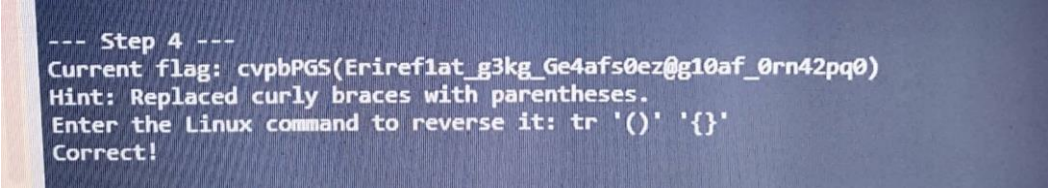
The hint said "Replaced underscores with dashes." Since underscores had been turned into dashes, I needed to turn the dashes back into underscores, so I used `tr '-' '_'`, which swaps every dash character back to an underscore character. Correct again.

```
--- Step 3 ---
Current flag: cvpbPGS(ErIref1at-g3kg-Ge4afs0ez@g10af-0rn42pq0)
Hint: Replaced underscores with dashes.
Enter the Linux command to reverse it: tr '-' '_'
Correct!
```

Step 4

Undo the brace/parenthesis swap

The hint said "Replaced curly braces with parentheses." So I needed to turn the parentheses back into curly braces, using `tr '()' '{}'` — matching each open/close parenthesis back to its curly-brace equivalent. That passed as well.

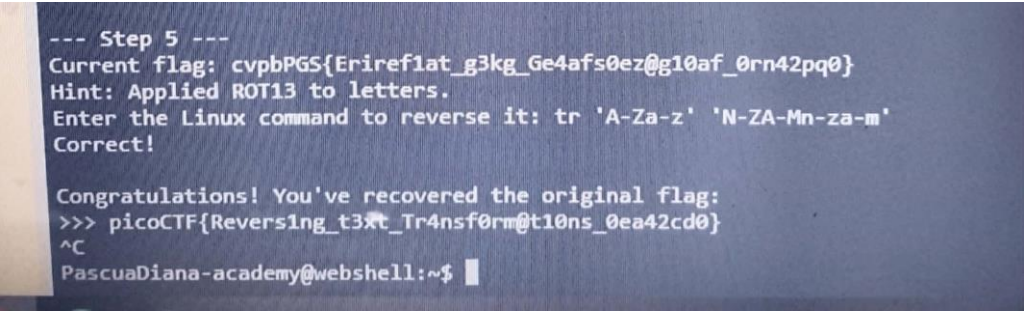
A terminal window with a dark blue background and white text. It shows the progress of a CTF challenge. The current flag is a base64-encoded string. The hint indicates that curly braces were previously replaced with parentheses. The user is prompted to enter a Linux command to reverse this transformation, and the correct command is provided.

```
--- Step 4 ---  
Current flag: cvpbPGS(Erireflat_g3kg_Ge4afs0ez@g10af_0rn42pq0)  
Hint: Replaced curly braces with parentheses.  
Enter the Linux command to reverse it: tr '()' '{}'  
Correct!
```

Step 5

Undo the ROT13 shift

The last hint said "Applied ROT13 to letters." ROT13 shifts every letter 13 places through the alphabet, and it's its own inverse, so running it again undoes it. I used `tr 'A-Za-z' 'N-ZA-Mn-m'`, which is the standard way to do a ROT13 substitution with `tr`. The server confirmed it was correct and printed "Congratulations! You've recovered the original flag," followed by the flag itself.

A terminal window with a dark blue background and white text. It shows the final steps of the CTF challenge. The current flag is a ROT13-encoded string. The hint indicates that ROT13 was applied to the letters. The user is prompted to enter a Linux command to reverse this transformation, and the correct command is provided. After successful execution, a congratulatory message is displayed, followed by the original flag.

```
--- Step 5 ---  
Current flag: cvpbPGS{Erireflat_g3kg_Ge4afs0ez@g10af_0rn42pq0}  
Hint: Applied ROT13 to letters.  
Enter the Linux command to reverse it: tr 'A-Za-z' 'N-ZA-Mn-m'  
Correct!  
  
Congratulations! You've recovered the original flag:  
>>> picoCTF{Revers1ng_t3xt_Tr4nsf0rm@t10ns_0ea42cd0}  
^C  
PascuaDiana-academy@webshell:~$
```

5. Flag

picoCTF{Revers1ng_t3xt_Tr4nsf0rm@t10ns_0ea42cd0}

6. Key Concept / What I Learned

This challenge was less about cracking anything and more about knowing basic Linux text-processing commands (`base64`, `rev`, `tr`) and understanding that most simple text transformations are reversible if you know exactly what was done. It reinforced how useful `tr` is for character-by-character substitution ciphers like ROT13 or simple swaps, and how important it is to read each transformation carefully rather than assume the previous step's fix will work again.

7. References

GNU Coreutils manual, tr command documentation — https://www.gnu.org/software/coreutils/manual/html_node/tr-invocation.html
